

REWEAR-FUSED — Galaxy-Tier Frontend Technical Specification (Mid-2026)

TL;DR

- **Recommended stack:** Next.js 16.2.6 (App Router, Turbopack) + React 19.2 + TypeScript 5.7 + Tailwind CSS v4.3 (Oxide) + React Three Fiber 9.6.1 (with three.js r184 WebGL2, plus a one-canvas WebGPU+TSL experiment) + Mol* (molstar) 5.9.0 for publication-grade PDB rendering + GSAP 3.15 (now fully free) with ScrollTrigger/ScrollSmoother + Motion 12.40 for React micro-interactions + Lenis 1.3.23 smooth scrolling + visx 3.12 for the bespoke trade-off curve and ECharts 6.1 for the compliance dashboard + Base UI 1.5 (via shadcn/ui 4.x) headless primitives + Söhne (Klim) + Editorial New (Pangram Pangram) + JetBrains Mono typography + Phosphor Icons 2.1.10 — deployed to Vercel with a recorded fallback video in case of demo Wi-Fi failure.
 - **Shock-factor combination:** Mol* protein cartoon/surface rendered against an HDRI dark studio + bloom + DOF + SSAO post-stack, with a GSAP ScrollTrigger-driven camera fly-through that hands off to a TSL compute shader particle storm (~250k particles) "depolymerizing" the garment mesh and re-spinning into a new garment — synchronized to Lenis smooth scroll and orchestrated with a single GSAP master timeline. No live GPU calls; everything reads from pre-computed JSON + PDB artifacts.
 - **Honest 72-hour realism:** a strong solo frontend dev can ship the scroll narrative, Mol* enzyme viewer, segmented-polymer R3F scene, postprocessing-driven glow look, compliance dashboard, and one hero closed-loop animation. The WebGPU/TSL compute particle path is **aspirational** — keep the safer WebGL2 (`GPUComputationRenderer`) build (~150k particles) as the demo-day default and ship WebGPU as a toggle if time permits.
-

Key Findings (at a glance)

1. **Next.js 16 + React 19.2 is the strongest base** for this project — R3F v9.6.1, Motion 12.40, and shadcn/ui 4.x all explicitly target React 19, and Next 16 makes Turbopack default for both dev and prod.
2. *Mol is the only realistic choice** for publication-grade PDB rendering in React — it's the engine PDBe and RCSB.org use in production. (`npm`)
3. **GSAP went 100% free** (including ScrollTrigger, ScrollSmoother, SplitText, MorphSVG, Flip) on **April 30, 2025** after Webflow's acquisition. This single change unlocks the most powerful scroll-narrative toolkit in the industry for a hackathon team at zero license cost.
4. **Tailwind CSS v4's Oxide engine** delivers a verified **182× incremental rebuild speedup** in the no-new-CSS case (35ms → 192μs, per Tailwind Labs internal benchmarks reported in

the v4.0 Medium post), and v4's CSS-first `@theme` config with OKLCH colors maps perfectly onto the dark-luminous accent system this brief calls for. `Medium`

5. **WebGPU is officially cross-browser Baseline as of January 13, 2026**, but for a 72-hour demo on uncertain hardware, **WebGL2 + three.js `GPUComputationRenderer` is the safe path and WebGPU/TSL is the aspirational toggle.**
 6. **Base UI 1.0 stable shipped December 2025** under the new `@base-ui/react` package name; shadcn/ui 4 supports it as a first-class registry alongside Radix. For new projects in 2026, Base UI is the modern recommendation. `Preblocks`
 7. **Phosphor over Lucide** — Lucide is everywhere via shadcn defaults; using Phosphor's six-weight family (especially duotone) is one of the cheapest ways to differentiate the look. `PkgPulse`
-

PART 1 — Core Framework & Rendering Foundation

(a) Framework choice — Next.js 16 (App Router) wins

Recommendation: Next.js 16.2.6 (latest stable, released May 7, 2026), App Router, with Turbopack as the default bundler. Next.js 16 ships Turbopack as the default for both dev and production builds and graduates Cache Components / PPR ("use cache" directive) — significantly faster cold-start and incremental builds than Next 15 (<https://nextjs.org/blog>). React 19.2 is the matched runtime. `Next.js` `Gigson`

Tradeoffs vs. the alternatives:

Framework	Verdict for REWEAR-FUSED	Why
Next.js 16.2 ✓	Pick this	Best React 19 + R3F v9 ecosystem fit; mature <code>next/dynamic</code> SSR-disabled loading for Mol*/R3F; tight Vercel deploy; massive shadcn/ui registry support
Astro 6.x	Strong second	Lowest JS payload, but R3F + Mol* are React-heavy "islands" that lose Astro's main advantage. Cloudflare officially acquired The Astro Technology Company on January 16, 2026 , <code>DEV Community</code> per Cloudflare's press release ("Cloudflare, Inc. (NYSE: NET) ... announced that The Astro Technology Company team, the creators of the Astro web framework, will be joining Cloudflare", cloudflare.com/press). <code>Cloudflare</code> For a <i>single</i> , animation-heavy product page Next.js is easier to wire end-to-end
SvelteKit	No	World-class compiler but no first-class R3F equivalent; ecosystem for Mol* (React UI) is poor
Remix / React Router 7	No	Remix merged into React Router 7; <code>DEV Community</code> less polished for static-artifact single-product showcases
TanStack Start	No	Excellent for type-safe data apps; not the win for static + heavy 3D demos

Cite: Next.js release blogs — <https://nextjs.org/blog/next-15> (Oct 2024 GA), <https://nextjs.org/blog/next-15-1>, and the running <https://nextjs.org/blog/index> (Next.js 16 Dec 2025; 16.1 Dec 18 2025; 16.2 Mar 2026).

(b) React 19 features that matter here

- **React Compiler (stable in React 19.2):** removes the need for hand-written `useMemo`/`useCallback` in the heavy scroll/animation tree — keep it on. (<https://nextjs.org/blog/next-15-1> — "React Compiler in Next.js with `experimental.reactCompiler`") `Next.js`
- **`use()` for promises:** lets us suspend on the static PDB fetch trivially. Read PDB via `use(fetch('/data/enzyme.pdb').then(r=>r.text()))` inside a server component, hand off to client.
- **Actions** — not relevant; no live server mutations.

- **Async params/searchParams (Next 15+ breaking change)** — relevant if you parameterize the gauge dashboard by query string.
- React 19.2 also unlocked the **Activity** API (R3F v9.5+ compatible — <https://github.com/pmndrs/react-three-fiber/releases>) which lets us pre-mount but pause off-screen 3D scenes — perfect for the multi-stage closed-loop sequence. [GitHub](#) [SourceForge](#)

(c) TypeScript setup (2026)

- TypeScript **5.7+** (Next 16 ships typed routes via `next typegen` — set `"experimental": { "typedRoutes": true }`). Use `"moduleResolution": "bundler"`, `"verbatimModuleSyntax": true`, `"noUncheckedIndexedAccess": true`. Pair with **Biome 2.x** or **ESLint 9 flat config** (Next 16 deprecated `next lint`). For 3D code add `@types/three` and `@types/webxr`.

(d) Build tooling — Turbopack is now default

Use Turbopack — production builds (`next build --turbopack`) went stable in Next 15.5 (<https://nextjs.org/blog>) and are the default in 16. Vite 6/7 (Vite is at 7.x as of 2026) is only relevant if you abandon Next; for an isolated R3F sandbox or Storybook, **Vite 7** is fine, otherwise stay on Turbopack. [Next.js](#)

(e) Styling — Tailwind CSS v4.3

Use Tailwind CSS v4.3.0 (released May 8, 2026) with the **Oxide engine** (Rust-rewritten core). Per Tailwind Labs' own benchmarks reported in the v4.0 Medium post, full builds are 3.5-5× faster and **incremental rebuilds with no new CSS go from 35ms to 192µs — a 182× speedup** (medium.com/@onix_react/what-to-expect-from-tailwind-css-v4-0). v4 also moves to **CSS-first config via `@theme` in your global CSS** (no more `tailwind.config.js`) — <https://tailwindcss.com/blog/tailwindcss-v4-alpha>; releases: <https://github.com/tailwindlabs/tailwindcss/releases>). Tailwind v4 requires **Safari 16.4+** / **Chrome 111+** / **Firefox 128+** because it uses `@property` and `color-mix()` natively — perfect for a controlled demo laptop. Use **OKLCH** colors in `@theme` for the bio-green/cyan and amber/coral accents so opacity-mixing stays in perceptually uniform color space. [Medium + 4](#)

Reject: vanilla-extract / Panda CSS / StyleX add bundling complexity that's not worth the 72-hour timeline. CSS Modules is fine for one-off shader containers but Tailwind is the workhorse.

PART 2 — 3D Molecular & Scientific Visualization (The Shock Factor)

(a) PDB rendering — *Mol (molstar) wins**, hands down

Use `molstar` 5.9.0 (latest on npm, <https://www.npmjs.com/package/molstar>; GitHub <https://github.com/molstar/molstar>). Mol* is the engine PDBe and RCSB PDB.org themselves use in production (David Sehnal et al., *Nucleic Acids Research* 2021, <https://doi.org/10.1093/nar/gkab314>). It produces **publication-grade cartoon, surface, ball-and-stick, and licorice representations** with built-in residue selection and structure highlighting — the only realistic choice for visualizing the Ser-His-Asp catalytic triad of the designed carbamate hydrolase with a glowing "active site" highlight. `npm` `npmjs`

Library	Pick for REWEAR-FUSED?	Notes
<i>Mol (molstar)*</i> 	Centerpiece	Best quality, supports custom dark themes, can be controlled headlessly via <code>PluginContext</code>
PDBe-Mol*	Alternative wrapper	Easier API but less custom-theme control; use for prototyping then port
3Dmol.js	No	Lighter but visual quality below Mol*; no comparable surface generator
NGL Viewer	No	Older WebGL stack; less actively developed

React integration recipe (the clean one — do NOT use `molstar-react` 0.5.2 which has stale-instance bugs): create a thin client component that mounts Mol* imperatively in `useEffect`, owns a `PluginContext` ref, and disposes on unmount. Pattern verified by the molstar maintainers in issue #226 (<https://github.com/molstar/molstar/issues/226>):

```
tsx
```

```

'use client';
import { useEffect, useRef } from 'react';
import { createPluginUI } from 'molstar/lib/mol-plugin-ui';
import 'molstar/build/viewer/molstar.css';

export function EnzymeViewer({ pdbUrl }: { pdbUrl: string }) {
  const parent = useRef<HTMLDivElement>(null);
  useEffect(() => {
    let plugin: any;
    (async () => {
      plugin = await createPluginUI({ target: parent.current!, render: undefined });
      await plugin.builders.structure.parseAndLoadStructure({ url: pdbUrl, format: '
      plugin.canvas3d.setProps({ renderer: { backgroundColor: 0x0a0d12 } });
    })();
    return () => plugin?.dispose();
  }, [pdbUrl]);
  return <div ref={parent} className="relative w-full h-full" />;
}

```

Dynamic-import it from Next.js with `next/dynamic({ ssr: false })` to keep the ~2 MB Mol* bundle off the critical path.

(b) Custom 3D moments — React Three Fiber 9.6.1 + drei 10.7 + three.js r184

Versions (verified June 2026):

- `three` **0.184.0** (r184, released April 16, 2026) — <https://github.com/mrdoob/three.js/releases> `npmx`
- `@react-three/fiber` **9.6.1** (April 28, 2026, supports React 19.0-19.2 — <https://github.com/pmndrs/react-three-fiber/releases>) `GitHub`
- `@react-three/drei` **10.7.7** — <https://www.npmjs.com/package/@react-three/drei> `npm`
- `@react-three/postprocessing` **3.0.4** + `postprocessing` **6.39.1** — <https://www.npmjs.com/package/postprocessing> `CodeSandbox` `npm`
- (Skip `@react-three/rapier` — no physics needed)

R3F v9 pairs with React 19 (<https://r3f.docs.pmnd.rs/tutorials/v9-migration-guide>) and supports **async `gl` props** specifically so you can hand it a `WebGPURenderer` factory. In Next.js 16, mount the `<Canvas>` inside a `'use client'` component wrapped in `next/dynamic({ ssr: false })`. `GitHub + 2`

(c) The "glowing molecules against depth" recipe

This is the exact visual stack to use — every effect serves the aesthetic:

tsx

```
<EffectComposer multisampling={0} disableNormalPass={false}>
  <SSAO samples={31} radius={0.18} intensity={28} luminanceInfluence={0.6} color="bl
  <Bloom mipmapBlur intensity={1.4} luminanceThreshold={0.85} luminanceSmoothing={0.
  <DepthOfField focusDistance={0.018} focalLength={0.06} bokehScale={4} />
  <ChromaticAberration offset={[0.0006, 0.0006]} />
  <Vignette eskil={false} offset={0.25} darkness={0.85} />
  <ToneMapping mode={ToneMappingMode.ACES_FILMIC} />
</EffectComposer>
```

Make molecules glow by **lifting emissive material colors above 1.0** (e.g. `emissive={new THREE.Color('#36F1B4').multiplyScalar(2.5)}`) — Bloom is selective by default and only blooms HDR-range pixels (<https://react-postprocessing.docs.pmnd.rs/effects/bloom>). Pair with `<Environment preset="studio" />` from drei for soft PBR reflections and `<MeshTransmissionMaterial>` for the "frozen-glass elastane filament" segments (segmented-polymer view). (Pmnd)

For the volumetric "atmospheric depth" feel: stack a `<Sparkles>` (drei) particle layer at low density behind the molecule with a 6–10% opacity gradient mesh as a "fog" — cheaper than true volumetric and visually equivalent at demo distance.

Custom GLSL: write one **screen-space god-rays** shader for the closed-loop moment (drei's `<GodRays>` works, but a custom radial-blur post-pass tied to the enzyme position reads more "instrument" than "anime").

(d) WebGPU vs WebGL2 in mid-2026

WebGPU's cross-browser Baseline status was officially reached when Firefox 147 shipped on **January 13, 2026** (byteiota.com/webgpu-2026). The full support matrix: Chrome 113+, Edge 113+, Safari 26+ (September 2025), and Firefox — per web.dev's official "WebGPU now supported in all major browsers" post — **"WebGPU is available on Windows as of Firefox 141. WebGPU is available in macOS Tahoe 26 on ARM64 machines, as of Firefox 145"** (web.dev/blog/webgpu-supported-major-browsers). three.js's `WebGPURenderer` is no longer experimental; it's the explicitly-recommended path for new projects (<https://threejs.org/manual/en/webgpurenderer.html>) and TSL (Three.js Shading Language) is

the production shader path. R3F v9's `async(gl)` prop was added specifically to support it.

byteiota + 3

Recommendation: WebGL2 is the safe demo-day default; ship one WebGPU scene as a "next-gen visuals" toggle. The judges will be on a controlled laptop with a recent Chrome, so WebGPU will work — but if the demo machine surprises you, WebGPU forces `await renderer.init()` and an asynchronous mount path that's one more failure point. Use `forceWebGL: true` on the renderer when you need to ship a guaranteed-safe build. (Three.js Roadmap) (Maxime Heckel)

(e) GPGPU particles — the depolymerization storm

For the hero "garment → cloud of monomers → re-spun garment" sequence:

- **Safe path (WebGL2):** three.js `GPUComputationRenderer` addon — allocate RGBA-float position/velocity textures, ping-pong two render targets per variable each frame, write update logic in GLSL fragment shaders. The three.js docs explicitly describe this: "The renderer has actually two render targets per variable, to make ping-pong. Textures from the current frame are used as inputs to render the textures of the next frame." Comfortably handles **100k-500k particles** on modern hardware (<https://threejs.org/docs/#examples/en/misc/GPUComputationRenderer>; tutorial: <https://tympanus.net/codrops/2024/12/19/crafting-a-dreamy-particle-effect-with-three-js-and-gpgpu/>). (Three.js)
- **Aspirational path (WebGPU/TSL):** declare `instancedArray(N, 'vec3')` storage buffers and write a compute pass with `Fn(() => { ... }).compute(N)`, dispatched via `renderer.computeAsync(fn)`. Scales to **~1M particles** (official example: https://threejs.org/examples/webgpu_tsl_compute_attractors_particles.html; Wawa Sensei TSL lesson: <https://wawasensei.dev/courses/react-three-fiber/lessons/tsl-gpgpu>).

Drive the particle simulation from **the same scroll progress value** GSAP uses for the camera — turning the scroll bar into a literal "depolymerize → re-spin" lever creates the emotional peak.

PART 3 — Motion, Animation & Scroll

(a) The library mix

Use both GSAP and Motion — they are *complementary*, not competing.

- **GSAP 3.15.0 (April 13, 2026; ScrollTrigger + ScrollSmoother included)** for orchestrated scroll-driven narrative timelines, ScrollSmoother, SplitText (now free), and the hero closed-loop master timeline. **GSAP went 100% free including ALL Club plugins on April 30, 2025** after Webflow's acquisition of GreenSock — confirmed verbatim by Webflow's

announcement: "everyone — whether or not you're a Webflow customer — will be able to leverage all of GSAP's tools completely free of charge, including the previously paid Club plugins" (<https://webflow.com/blog/gsap-becomes-free>). CSS-Tricks summarized: "GSAP is Now Completely Free, Even for Commercial Use" (<https://css-tricks.com/gsap-is-now-completely-free-even-for-commercial-use/>). Includes ScrollTrigger, ScrollSmoother, SplitText, MorphSVG, DrawSVG, Flip, MotionPath. (Webflow + 2)

- **Motion 12.40 (motion.dev — formerly Framer Motion; published May 21, 2026 — <https://www.npmjs.com/package/motion>)** for React-idiomatic component-level micro-interactions: layout animations (`layout` prop), `AnimatePresence`, `whileHover`/`whileTap`, gesture handlers, the `<motion.div>` API. Motion 12 has full React 19 concurrent-rendering support and now supports oklch/oklab/lab/lch/color-mix interpolation natively. (npm) (Refine)
- Skip anime.js v4 and React Spring — GSAP + Motion covers their use cases cleanly.

(b) GSAP React integration + Lenis

Use the `@gsap/react` 2.1.x package and its `useGSAP()` hook — drop-in replacement for `useEffect`/`useLayoutEffect` that handles GSAP context creation and cleanup automatically (<https://gsap.com/resources/React/>). Register plugins once at module top: (npm)

```
tsx

'use client';
import gsap from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
import { ScrollSmoother } from 'gsap/ScrollSmoother';
import { SplitText } from 'gsap/SplitText';
import { useGSAP } from '@gsap/react';
gsap.registerPlugin(ScrollTrigger, ScrollSmoother, SplitText, useGSAP);
```

Lenis 1.3.23 (<https://github.com/darkroomengineering/lenis>) for buttery smooth scrolling. **Pair Lenis with GSAP ScrollTrigger** (do NOT also enable ScrollSmoother — pick one) by routing Lenis's `scroll` event into `ScrollTrigger.update` and Lenis's `raf` into `gsap.ticker`:

```
ts

const lenis = new Lenis({ autoRaf: false });
lenis.on('scroll', ScrollTrigger.update);
gsap.ticker.add((t) => lenis.raf(t * 1000));
gsap.ticker.lagSmoothing(0);
```

This is the darkroom.engineering / GSAP-recommended pattern (verbatim in Lenis README — <https://github.com/darkroomengineering/lenis>). (GitHub + 2)

(c) View Transitions API — production-ready, use sparingly

Same-document View Transitions became Baseline Newly Available on **October 14, 2025**, per web.dev's official Baseline post: "Firefox intends to include these features in the upcoming Firefox 144 release which becomes stable on October 14, 2025. This will make these features Baseline Newly available" (web.dev/blog/same-document-view-transitions-are-now-baseline-newly-available; cross-references https://developer.mozilla.org/en-US/docs/Web/API/View_Transition_API and <https://www.testmuai.com/learning-hub/view-transitions-api-browser-support/>). **Cross-document transitions** ship in Chrome/Edge 126+ and Safari 18.2+, **still not in Firefox** as of mid-2026 (<https://trade-assistance.com/blog/cross-document-view-transitions-mpa-2026/>). Use for the "Engine A → Engine B → Engine C" tab switch (give the panel a `view-transition-name`), wrap state change in `document.startViewTransition()`. Don't use for the hero animation — GSAP gives you total control there. (Chrome Developers + 2)

(d) Native CSS scroll-driven animations

`animation-timeline: scroll()` and `view()` are **fully supported in Chrome/Edge 115+, Safari 26 (Sept 2025), Opera 101+; Firefox is partial behind a flag** (<https://www.joshwcomeau.com/animation/scroll-driven-animations/>; <https://rebeccamdeprey.com/blog/scroll-driven-animations-css>). Use natively for **small reveal animations** (a fade-up paragraph, a number counter triggered by `view()`) — cheaper than GSAP and runs on the compositor thread. Reserve GSAP for the orchestrated multi-element scrubbed sequences. (Rebeccamdeprey) (DevToolbox)

PART 4 — Data Visualization

(a) Two libraries, two jobs

- **visx 3.12.0** (`@visx/xychart`) — for the **bespoke mechanics-vs-cleavability trade-off curve** in Engine A. visx gives you D3-powered scales/axes/shapes as low-level React primitives, so we can render the Pareto frontier with a glowing accent stroke, animated tooltip, and custom annotation labels in the editorial-science aesthetic (<https://github.com/airbnb/visx>; LogRocket 2026 review: "visx gives you scales, axes, shapes, groups, and utilities for building your own chart components"). Use `@visx/curve` (`curveCatmullRom`) + `@visx/gradient` for the line glow. (npm) (LogRocket)

- **ECharts 6.1** (`echarts` 6.1.0, May 2026, with `echarts-for-react` wrapper) — for the **Engine C compliance dashboard gauges**. ECharts has gauge, radial bar, sunburst, and treemap visualizations built-in with declarative options — fastest path to a premium-looking regulatory dashboard in 72 hours.

Reject: Recharts 3.8 (too pre-styled, too SaaS-default); Nivo (heavyweight, slower SSR); Tremor (Tremor is built on Recharts; same problem); D3 raw (3× the code for the same result as visx); Observable Plot (no React idiom).

(b) Premium numbers and gauges

- **Animated number counters** — use Motion's `useMotionValue` + `useTransform` driving a `<motion.span>` updated by `useSpring` for that smooth "instrument readout" tick. Trigger via `whileInView`.
 - **Gauges** — ECharts `series.type: 'gauge'` with `axisLine.lineStyle.color` as an OKLCH gradient (bio-green → amber when crossing compliance threshold), and a custom CSS-clipped halo behind it for the glow.
 - **Progress indicators** — pure CSS `conic-gradient()` rings with `@property --p` declared in Tailwind v4's `@theme` for transition-able percentage values.
-

PART 5 — UI Component Layer, Typography & Polish

(a) Headless primitives — Base UI via shadcn/ui registry

Use shadcn/ui 4.x (CLI v4 / `shadcn create`) — released March 2026 —

<https://ui.shadcn.com/docs/changelog>) with the Base UI base, not Radix. The story changed materially in 2025–26:

- **December 2025:** Base UI hit **v1.0 stable** (now published under `@base-ui/react` 1.5.0 as of May 19, 2026 — <https://www.npmjs.com/package/@base-ui/react>), built by the same team that originally created Radix Primitives + the MUI team. (Preblocks + 2)
- **December 2025:** shadcn/ui added a first-class Base UI registry (<https://ui.shadcn.com/docs/changelog>): "We rebuilt every component for Base UI, keeping the same abstraction. They are fully compatible with your existing components, even those pulled from remote registries." (Preblocks)
- Base UI includes built-in **multi-select**, **combobox**, **autocomplete** that Radix lacks, and uses a cleaner `render` prop pattern instead of `asChild`. (Preblocks) (Smithery)
- Radix's parent WorkOS has under-invested since acquisition, with growing maintenance backlog (<https://preblocks.com/blog/radix-ui-vs-base-ui>). (Preblocks)

Concrete setup: `(npx shadcn@latest create)` → choose **Base UI**, style **Rhea** (most compact — best for an instrument-dense interface), icon set **Phosphor**, base color custom (define in `@theme`).

(b) Typography — distinctive, non-generic, "instrument meets editorial"

The three-face system (no Inter, no Roboto, no Arial, no Space Grotesk):

Role	Typeface	Foundry	License	Why
Display / brand	Editorial New	Pangram Pangram	Free w/ basic license	Premium high-contrast didone-style serif; "editorial science journal" voice. https://pangrampangram.com/
Body / UI	Söhne (Klim) — or General Sans (Pangram Pangram, free) as the budget swap	Klim Type Foundry	Paid (Söhne) / Free (General Sans)	Söhne is the contemporary neo-grotesque verified in use at Stripe (Design Blog) — Fonts In Use documents Stripe's 2020 redesign "replacing Camphor in favor of Klim Type Foundry's Söhne" (Fonts In Use) (fontsinuse.com/uses/35338/stripe-website-2020) — and at OpenAI from February 28, 2023 , which "replaced the long-running Charter and Colfax with two Klim Type Foundry typefaces: Signifier and Söhne" (Fonts In Use) (fontsinuse.com/uses/52248/openai-website). General Sans gives 80% of the feel for free
Mono / data / SMILES	JetBrains Mono	JetBrains	Free, OFL	Tall x-height, technical personality, (Design Blog) excellent for residue labels (Ser-195, His-57, Asp-102) and SMILES strings

For a more avant-garde display option, **ABC Diatype Mono** (Dinamo) or **Reckless Neue** (Displaay) elevate further at cost.

Self-host with `(next/font/local)` in Next 16 — drop variable font files into `(/app/fonts)`, declare in a layout, and reference via CSS variables (`(--font-display)`, `(--font-body)`, `(--font-mono)`). Use **variable fonts** so you can animate weight via Motion / GSAP on scroll-reveal headlines.

(c) Iconography — Phosphor Icons 2.1.10

`(@phosphor-icons/react)` 2.1.10 (<https://www.npmjs.com/package/@phosphor-icons/react>) — 9,000+ icons in **six weights** (thin, light, regular, bold, fill, duotone). The variety + the **duotone**

weight is what differentiates the UI from the generic Lucide-everywhere shadcn aesthetic (<https://hugeicons.com/blog/design/8-lucide-icons-alternatives-that-offer-better-icons>). Use thin/light weights for chrome, bold/fill for active states, duotone for the hero feature illustrations. Lucide stays available as the shadcn default — use it for shadcn-internal component icons only.

(npm + 2)

(d) The "clean but shocking" balance — design-system principles

A 4-rule discipline that prevents heavy 3D from becoming cluttered:

1. **One 3D moment per viewport, max.** The molecule or the chart, never both side-by-side.
2. **Negative space as the primary layout primitive.** Use **8-point spacing scale** in Tailwind v4 (`@theme { --spacing: 0.25rem }`) and never use the dense end. Hero sections: 200px+ vertical padding; never less than 64px between sections.
3. **Two-accent rule:** bio-green/cyan (`--accent-bio: oklch(0.83 0.18 175)`) ONLY for enzyme/biology contexts; amber/coral (`--accent-fiber: oklch(0.74 0.17 45)`) ONLY for fiber/material contexts. The third "color" is white-at-low-opacity text on a near-black (`oklch(0.16 0.01 240)`) backdrop. No other hues allowed except in data viz scales.
4. **Token architecture (Tailwind v4 CSS-first):**

CSS

```
@theme {  
  --color-bg: oklch(0.13 0.01 240);  
  --color-bg-elevated: oklch(0.17 0.01 240);  
  --color-fg: oklch(0.96 0.005 240);  
  --color-fg-muted: oklch(0.65 0.01 240);  
  --color-accent-bio: oklch(0.83 0.18 175);  
  --color-accent-fiber: oklch(0.74 0.17 45);  
  --color-rule: oklch(0.25 0.005 240 / 0.6);  
  --font-display: 'Editorial New', serif;  
  --font-body: 'Söhne', system-ui, sans-serif;  
  --font-mono: 'JetBrains Mono', monospace;  
  --shadow-bloom: 0 0 80px -8px oklch(0.83 0.18 175 / 0.45);  
}
```

Every component reads only from these tokens. To reskin, change tokens — never component code.

(a) Performance discipline for heavy WebGL + Mol*

- **Code-split aggressively:** `next/dynamic(() => import('@components/EnzymeViewer'), { ssr: false, loading: () => <Skeleton/> })` for Mol*, each R3F scene, and the dashboard charts. Mol* alone is ~2 MB gzipped.
- **R3F** `frameLoop="demand"` for static scenes (the segmented-polymer hero with no idle motion) and `frameLoop="always"` only during active animations — call `invalidate()` from GSAP onUpdate. Saves 60% GPU during scroll between scenes.
- **React Suspense** + `Activity` wrap each Engine panel so off-screen scenes pause instead of being torn down.
- **Asset compression:** any glTF assets through `gltfpack` + **Meshopt** (preferred in 2026 over Draco for browser decode speed) + **KTX2** Basis-compressed textures.
- **PDB files:** pre-convert to BinaryCIF (.bcif) via Mol*'s `cif2bcif` CLI — 80% smaller, instant load.
- **HDRI:** use a 1k or 2k `.hdr`, never 4k+; preload with drei `useEnvironment`.
- **Preload during idle:** `<link rel="prefetch">` the next scene's bundle from `useIdleCallback`.

(b) Deployment — Vercel for this project

Use **Vercel**. The Hobby tier (<https://agentdeals.dev/hosting-free-tier-comparison-2026>) gives 100 GB bandwidth, 1M function invocations, and Next.js 16-native ISR / Turbopack — Vercel built Next.js, so deploys are zero-friction. For a hackathon, that's all you need. `Agentdeals`

- Cloudflare Pages is cheaper at scale but Next 16 SSR support requires `@opennextjs/cloudflare` adapter — extra config you don't want at hour 60. `Vibe Coder Blog`
- Netlify is fine but no integration advantage.

Demo-safety protocol (mandatory):

1. `next build && next start` locally on the demo laptop the night before — verify offline-localhost works.
2. Set up a local Wi-Fi hotspot from a backup phone, not the venue's network.
3. **Record a 4K screen capture of the full demo flow** (OBS, 60fps) the morning of, in case all else fails. Have the file open in QuickTime/VLC ready to fullscreen as a fallback.
4. Cache the deployed Vercel URL in the browser, then **also** keep `localhost:3000` running in a second tab as a backup.
5. Disable Chrome extensions; use a clean profile.
6. Test on the actual demo display resolution (usually 1080p or 4K projector).

(c) Loading states matching the aesthetic

Skip the spinner. Use:

- *Mol and R3F scene initialization*:* a thin animated rule of monospaced data — "INITIALIZING SCENE • LOADING 1LOL.bcif • COMPILING SHADERS" with a JetBrains Mono ticker driven by Motion's `useMotionValue`. Reads as "scientific instrument warming up."
 - **Skeleton blocks**: match the final layout exactly (use `<Skeleton>` from shadcn but restyle with `bg-[var(--color-bg-elevated)]` and a subtle OKLCH-gradient shimmer at 12% opacity over 1.4s).
 - **Preloader**: on first page load, hold a `position: fixed` overlay with the logo + a typographic countdown ("99% • PRE-COMPUTING PASSPORT...") that animates with `SplitText`, lifted off-screen with a GSAP timeline when the first 3D scene's `onCreated` fires.
-

Recommendations

The Single "Galaxy-Tier" Stack — concrete `package.json` excerpt

```
jsonc
```

```

{
  "next": "16.2.6",
  "react": "19.2.6",
  "react-dom": "19.2.6",
  "typescript": "5.7.x",
  "tailwindcss": "4.3.0",
  "@tailwindcss/vite": "4.3.0",

  "three": "0.184.0",
  "@react-three/fiber": "9.6.1",
  "@react-three/drei": "10.7.7",
  "@react-three/postprocessing": "3.0.4",
  "postprocessing": "6.39.1",
  "molstar": "5.9.0",

  "gsap": "3.15.0",
  "@gsap/react": "2.1.2",
  "motion": "12.40.0",
  "lenis": "1.3.23",

  "@visx/xychart": "3.12.0",
  "echarts": "6.1.0",
  "echarts-for-react": "3.0.2",

  "shadcn": "4.9.0",
  "@base-ui/react": "1.5.0",
  "@phosphor-icons/react": "2.1.10",
  "lucide-react": "1.17.0",
  "class-variance-authority": "0.7.x",
  "tailwind-merge": "2.x",

  "@react-three/offscreen": "0.0.x"
}

```

The 5 "Shouldn't Be Possible in 72 Hours" Moments

1. **Open with shock:** Hero is a Mol* enzyme rendered in dark studio HDR, slowly rotating, with an OKLCH cyan glow around the Ser-His-Asp active site driven by a custom Mol* `repr-builder` overlay. Headline in Editorial New italic 96px sits in the upper third. Reads as a *Nature* cover.
2. **Scroll-pinned camera through the molecule:** GSAP ScrollTrigger pins the canvas and scrubs the camera *through* the enzyme into the substrate-binding cleft as the user scrolls.

Substrate molecule (cleavable elastane fragment) materializes from particles into the active site. The Sci-Vis equivalent of an Apple product page.

3. **Cross-scene morph at Engine A → B handoff:** Use View Transitions API with a shared `view-transition-name` on the molecule canvas, plus an instant GSAP fade-to-particles. Reads as if the molecule itself is reconfiguring.
4. **The closed-loop emotional peak:** GPGPU particle system (250k particles on WebGL2; toggle to 1M on WebGPU/TSL on capable hardware) — a child's garment mesh, sampled to particle positions via `SamplerMesh`, dissolves into a swirling cloud as a substrate-bound enzyme animation plays, then re-coalesces into a *different* garment. Driven by a single scroll-progress value tied to all three scenes simultaneously. Bloom + DOF + god rays + a chromatic-aberration micro-pulse on the re-coalescence frame.
5. **Compliance dashboard reveal:** All gauges count up from 0 in choreographed stagger (Motion's `useInView` + staggered delays); the final gauge crosses into the "compliant" zone and triggers a single SplitText "PASSPORT VERIFIED" reveal — the ONLY moment of UI text animation, making it land.

Staged Build Plan (72 hours, one strong frontend dev)

Hours	Stage	Drop-if-late threshold
0-6	Repo, Next 16 + Tailwind v4 + shadcn/Base UI + typography + tokens + layout shell	Cannot drop — foundation
6-14	Scroll-driven landing with Lenis + GSAP ScrollTrigger, hero molecule placeholder, scene-stitching plan	Cannot drop — narrative spine
14-24	Mol* enzyme viewer + custom dark theme + active-site highlight + scroll-camera scrub	Cannot drop — the centerpiece
24-36	R3F segmented-polymer scene + postprocessing stack + Environment + bloom tuning	Cannot drop — second visual peak
36-48	Closed-loop animation: GPGPU particles (WebGL2 path), garment morph, single scroll-driver	Reduce particle count to 30k if behind
48-60	Compliance dashboard (ECharts gauges) + trade-off curve (visx) + Motion counters	Drop trade-off curve animations if behind, keep static
60-66	Performance pass: code-splitting, frameloop tuning, .bcif conversion, Lighthouse	Cannot drop
66-72	OBS recording, demo rehearsal, fallback testing, Vercel deploy verification	Cannot drop

Stretch toggles (only if hours 60–66 finish early): WebGPU/TSL compute particle path; custom screen-space god-rays GLSL; Cross-document View Transitions for Engine A/B/C navigation.

Caveats

- *Mol version verification:** the molstar npm package was at v5.9.0 in mid-2026 search results; if `(npm view molstar version)` returns a different number at build time, use whatever is latest stable. The integration recipe is version-stable.
- **GSAP licensing:** GSAP became fully free for commercial use on April 30, 2025, per Webflow's official announcement (<https://webflow.com/blog/gsap-becomes-free>) and CSS-Tricks (<https://css-tricks.com/gsap-is-now-completely-free-even-for-commercial-use/>). Some old guides still tell you SplitText/ScrollSmoother require Club GSAP — they do not, as of 2025. (Alijavadi)
- **React 19 + R3F minor versions:** R3F v9 bundles its own reconciler to support React 19.0 through 19.2 (<https://github.com/pmndrs/react-three-fiber/releases> v9.5.0 notes). Pin React to a single 19.x patch and `(@react-three/fiber)` to 9.6.1 to avoid mismatch debugging. (GitHub) (SourceForge)
- **WebGPU on demo hardware:** verify the demo laptop's Chrome version supports WebGPU before committing to it for the hero scene. If unknown, ship WebGL2 — visual difference at demo distance is negligible.
- **Base UI is newer than Radix:** if something feels under-documented, fall back to Radix via the shadcn Radix registry — they share the abstraction so switching is per-component, not project-wide.
- **No live GPU calls:** the spec assumes Engines A/B/C read pre-computed static JSON + PDB artifacts only. Never let a demo depend on network reachability.
- *Mol + Tailwind v4 CSS conflict:** Mol*'s shipped `(molstar.css)` uses classic CSS that may interact with Tailwind v4's reset. Scope it inside a `(data-molstar)` wrapper using v4's `(@layer)` directive to isolate.
- **Astro is a tempting alternative** if you're an Astro shop; with Cloudflare's January 16, 2026 acquisition (per cloudflare.com press release), Astro 6 deployment on Cloudflare Pages is genuinely seamless. But for this specific brief, the React-heavy Mol* + R3F ecosystem makes Next.js 16 the clearer win. (PkgPulse)